Module 5

Memory Management

Memory

- Memory consists of a large array of bytes, each with its own address.
- The CPU fetches instructions from memory according to the value of the program counter.
- The memory unit sees only a stream of memory addresses; it does not know how they are generated or what they are for.

Direct Access to Memory and Registers

- The CPU can directly access only two types of storage: **main memory and registers** built into the processing cores.
- Any data or instruction being used must be in one of these locations. If it is not, the data must be moved into memory for the CPU to access.
- Registers are extremely fast (accessible in one clock cycle), but memory access is slower as it requires communication over the memory bus. So, cache is used.

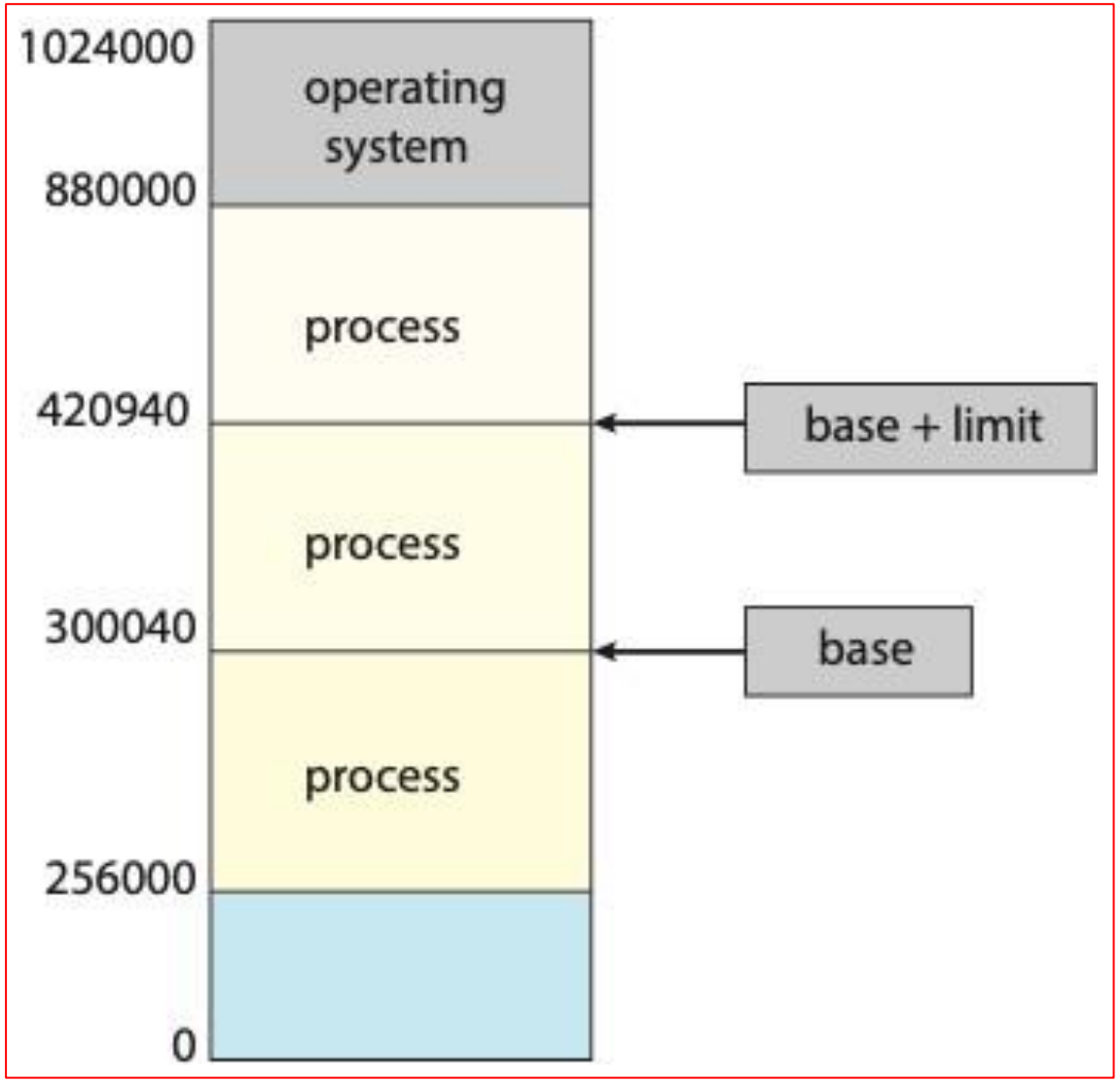
Hardware Memory Protection

- For proper operation, it's essential to protect the operating system from unauthorized access by user processes and to prevent user processes from accessing each other's memory.
- This protection is enforced by hardware using mechanisms like **base and limit registers.**
- Each process has a separate memory space with a range of legal addresses that the process may access.

Hardware Memory Protection

- **Base register** holds the **smallest legal memory address** for a process.
- **Limit register** specifies the **range of legal addresses**.
- The CPU compares every address generated in user mode against these registers.
- Any illegal access results in a trap to the operating system, which treats the attempt as a fatal error.

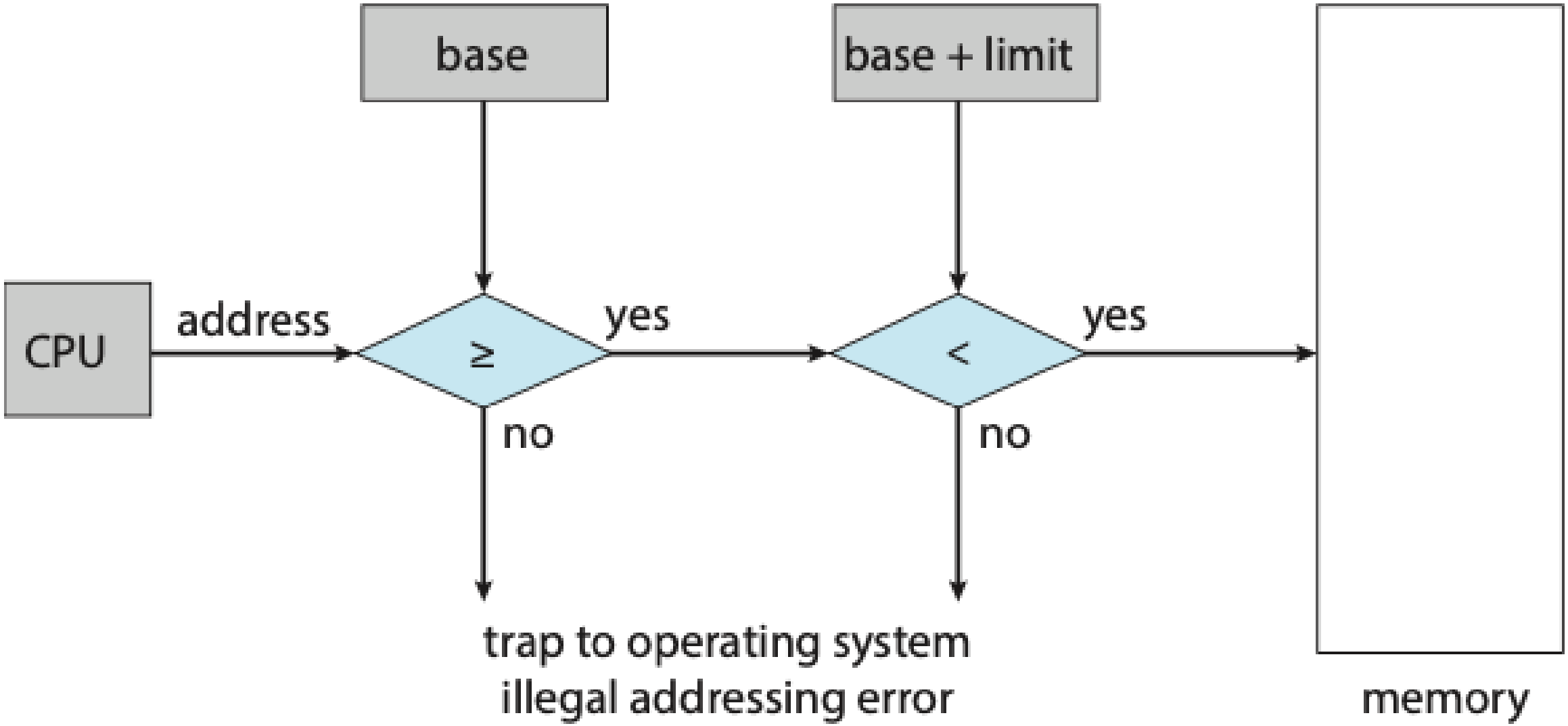
Hardware Memory Protection



Hardware Memory Protection

- **Base Address Register**: The starting memory address of a process in main memory. It defines the lowest legal memory address that the process can access.
- **Limit Register**: Specifies the size of the process or the range of addresses the process is allowed to access. The value in the limit register is typically the length of the addressable memory region for that process, starting from the base address.
- The base address tells where the process begins in memory.
- The limit tells how much memory (in terms of size) the process can access, thus defining the upper boundary of accessible memory as **base + limit**

Hardware Memory Protection



User and Kernel Mode

- The base and limit registers can be loaded only by the operating system, which uses a special privileged instruction.
- Protection mechanisms ensure that user processes cannot access kernel memory or manipulate hardware directly.
- Only the operating system can modify the value of base and limit registers, as this operation can only be performed in kernel mode. User programs cannot change these register values.

User and Kernel Mode

- The operating system, executing in kernel mode, is given unrestricted access to both operating-system memory and users' memory

Address binding - Process Execution

- Programs are stored as binary executable files on disk and need to be loaded into memory to run.
- The process accesses instructions and data from memory during execution.
- When process terminates, and its memory is reclaimed for use by other processes.

Address Binding

- Address Binding refers to the process of mapping logical (virtual) addresses to physical addresses in memory.
- It occurs in different stages during a program's lifecycle: **compile-time, load-time, and execution-time.**
- The main goal of address binding is to ensure that when a program accesses memory, it can **find the right data or instruction in physical memory.**

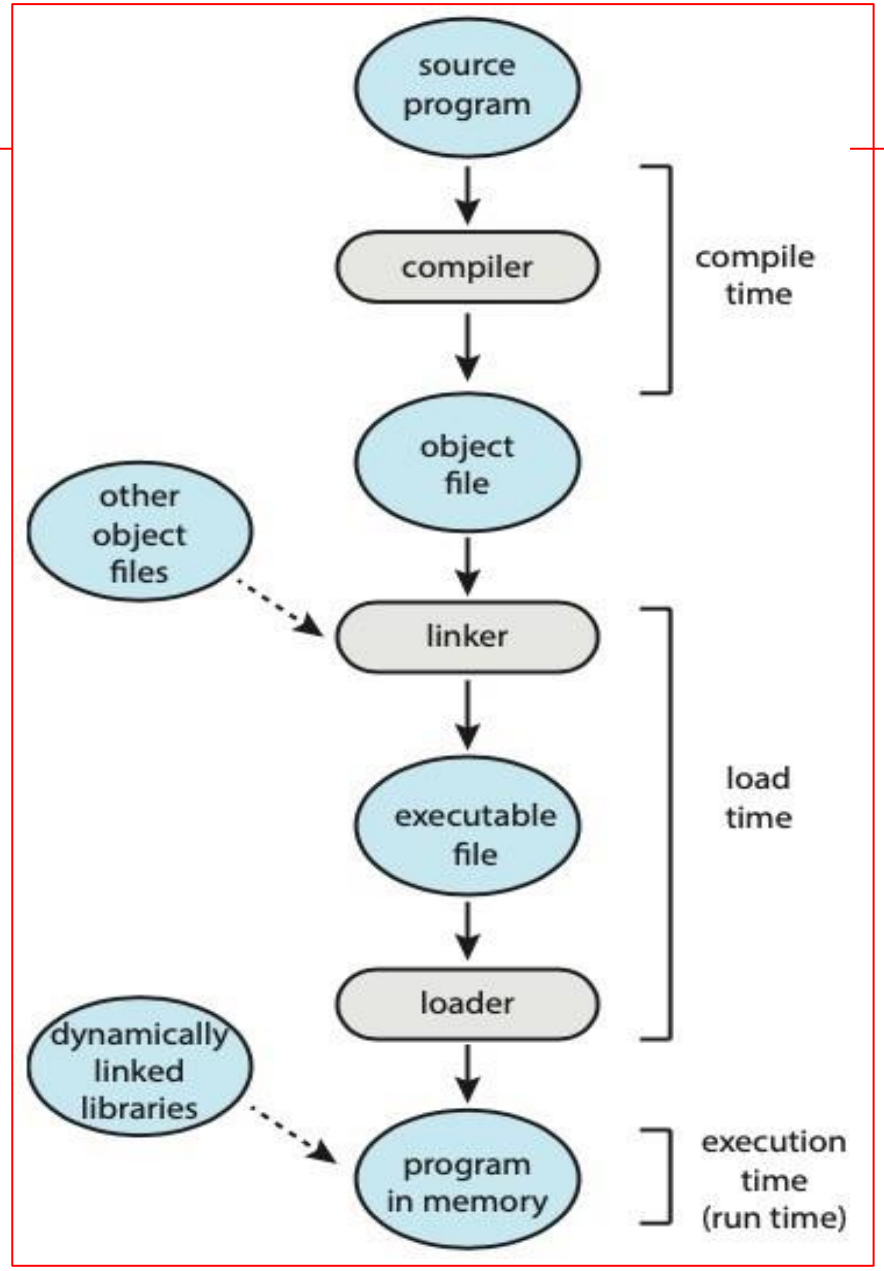
Address Binding

- In most cases, a user program goes through several steps - Addresses maybe represented in different ways:
- In a source code, symbolic addresses are identifiers for variables, functions, classes, and other constructs.
- A compiler typically binds these symbolic addresses to relocatable addresses
- The linker or loader in turn binds the relocatable addresses to absolute addresses.

Address Binding

- The binding of instructions and data to memory addresses can be done at any step along the way:
- Compile time. If you know at compile time where the process will reside in memory, then absolute code can be generated.
- Load time. If it is not known at compile time where the process will reside in memory, then the compiler must generate relocatable code.
- Execution time. If the process can be moved during its execution from one memory segment to another, then binding must be delayed until run time.

Address Binding



Address Binding - Compile-Time Address Binding

- Address Binding refers to mapping symbolic or relocatable addresses in the program to actual memory addresses
- The compiler determines where a program will reside in memory.
- The addresses in the program are set during compilation, and if the program's memory location needs to change later (e.g., if the program is loaded into a different memory location), it must be recompiled.

Address Binding - Compile-Time Address Binding

- Example
  - Suppose a small program is always loaded into memory starting at address 1000. The compiler generates machine code that expects the program to start at this address.
  - If the program is loaded elsewhere (e.g., address 2000), the code will not work unless it is recompiled to reflect this new memory location.

Address Binding - Load-Time Address Binding

- The exact memory location where the program will be loaded is not known at compile time.
- The linker or loader binds addresses when the program is loaded into memory. The addresses are relocatable, meaning they can adjust based on where the program is loaded in memory.

Address Binding - Load-Time Address Binding

Example

- The program is compiled with relocatable addresses, which say, "I need memory starting from some location." If the program is loaded into address 3000, the loader adjusts all addresses to work from that starting point.
- If the program is later loaded into address 5000, the loader adjusts the addresses accordingly, but no recompilation is needed.

Address Binding - Execution-Time Address Binding

- The program can be moved in memory during its execution (e.g., due to swapping or paging).

- The binding happens dynamically at run time, meaning the operating system keeps track of where the program is at any given time and translates addresses on-the-fly.

Address Binding - Execution-Time Address Binding

- Example
  - The program starts at address 4000, but during execution, the operating system moves part of it to address 7000 (perhaps due to memory management or multitasking).
  - Every time the program tries to access a memory location, the operating system translates its logical addresses to the current physical address in memory.

Address Binding - Examples

- Example
  - When you write code like int x = 5;, the compiler assigns an address to x, say 1000.
  - If the compiler knows that the program will always be loaded starting at memory location 4000, it can assign x a fixed address of 5000 (4000 + 1000).
  - This is_____?

Address Binding - Execution-Time Address Binding

- Example
  - Now, imagine the program might not always start at 4000. It could start at any available memory address.
  - The program is compiled with relocatable addresses. For example, x might be stored at offset 1000 relative to the program's starting address.
  - When the program is loaded into memory (e.g., starting at 8000), the loader will adjust the address of x to 9000 (8000 + 1000).

Address Binding - Execution-Time Address Binding

- Example
  - If your program is moved to another part of memory (e.g., because of paging or swapping), the operating system will adjust the memory addresses dynamically during execution.
  - For example, if the OS moves the program to 12000, the address of x will be updated to 13000 (12000 + 1000) while the program is running.

Logical Address Space

- A logical address is **an address generated by the CPU** during program execution.
- It is the **address used by a program to reference memory locations**.
- This **address is visible to the user or programmer** and is often seen in the program as **symbolic references** (like variables, arrays, etc.).
- Logical addresses are part of the logical address space (also known as the **virtual address space**), which is the set of all possible addresses that a program can generate
- The logical address **does not correspond directly to an actual location in physical memory**. It is translated into a physical address by the memory management unit (MMU) via address translation techniques like paging or segmentation.

Logical Address Space

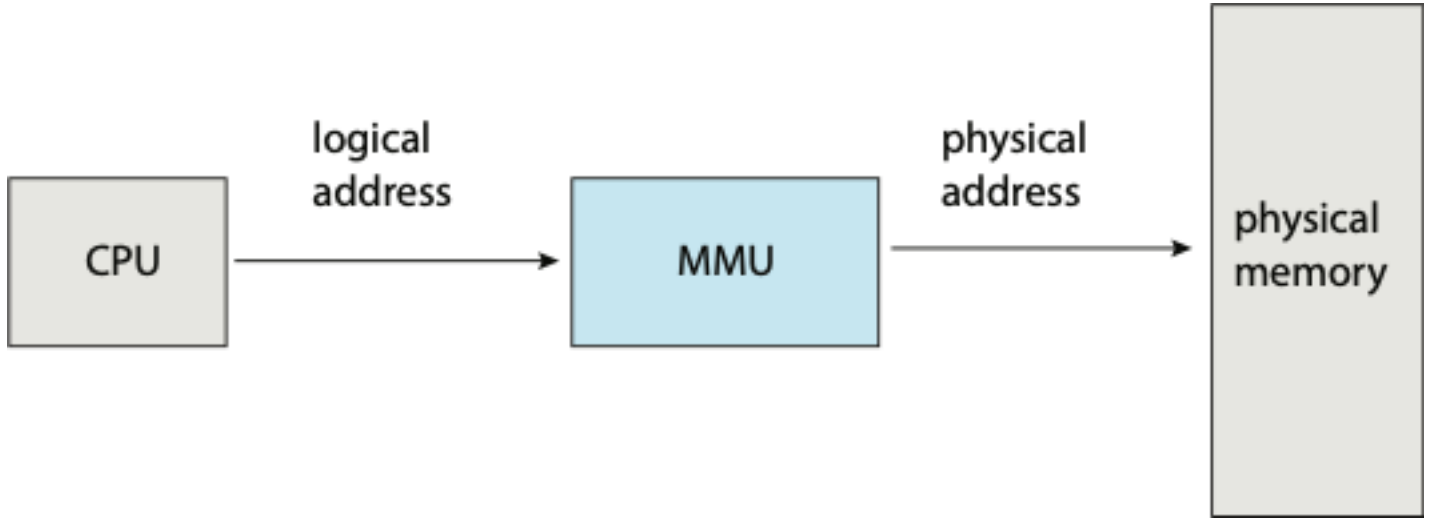
- **Why Use Logical Addresses?**
  - Logical addresses provide an extra layer of abstraction. This allows programs to run without needing to know exactly where their data is stored in physical memory.
  - It also gives flexibility to the operating system, allowing it to move programs around in memory without the programs needing to know.

Physical Address Space

- A physical address is the **actual address in the main memory** (RAM).
- It represents the **real location of data or instructions in physical memory** hardware.
- This address is **not visible to the user or programmer** and is only used by the hardware (the memory management unit, CPU, and OS).
- Physical addresses are part of the physical address space, which is the set of all physical memory locations in the hardware.
- The physical address is **obtained by translating the logical address** using the memory management unit (MMU).

Physical Address Space

- When a program is running, the CPU generates a logical address (e.g., 0x005A). This address is not the actual location in physical memory.
- The MMU translates this logical address to a physical address (e.g., 0xF12345), which is where the data is stored in the RAM.



Memory Management Unit (MMU)

- The MMU is a **special hardware component** in the computer that helps **translate the virtual (logical) addresses used by a program into physical addresses** that point to actual locations in the computer's memory (RAM).
- Think of the MMU as a translator that maps the addresses a program uses into real memory addresses behind the scenes

Memory Management Unit (MMU)

- When a **program is running**, it doesn't directly use the physical addresses where data is stored in memory. Instead, it uses virtual addresses, which are more flexible and can be different from the physical addresses.
- The MMU converts these virtual addresses to physical addresses so that the CPU can find the correct location in the actual memory.

Memory Management Unit (MMU)

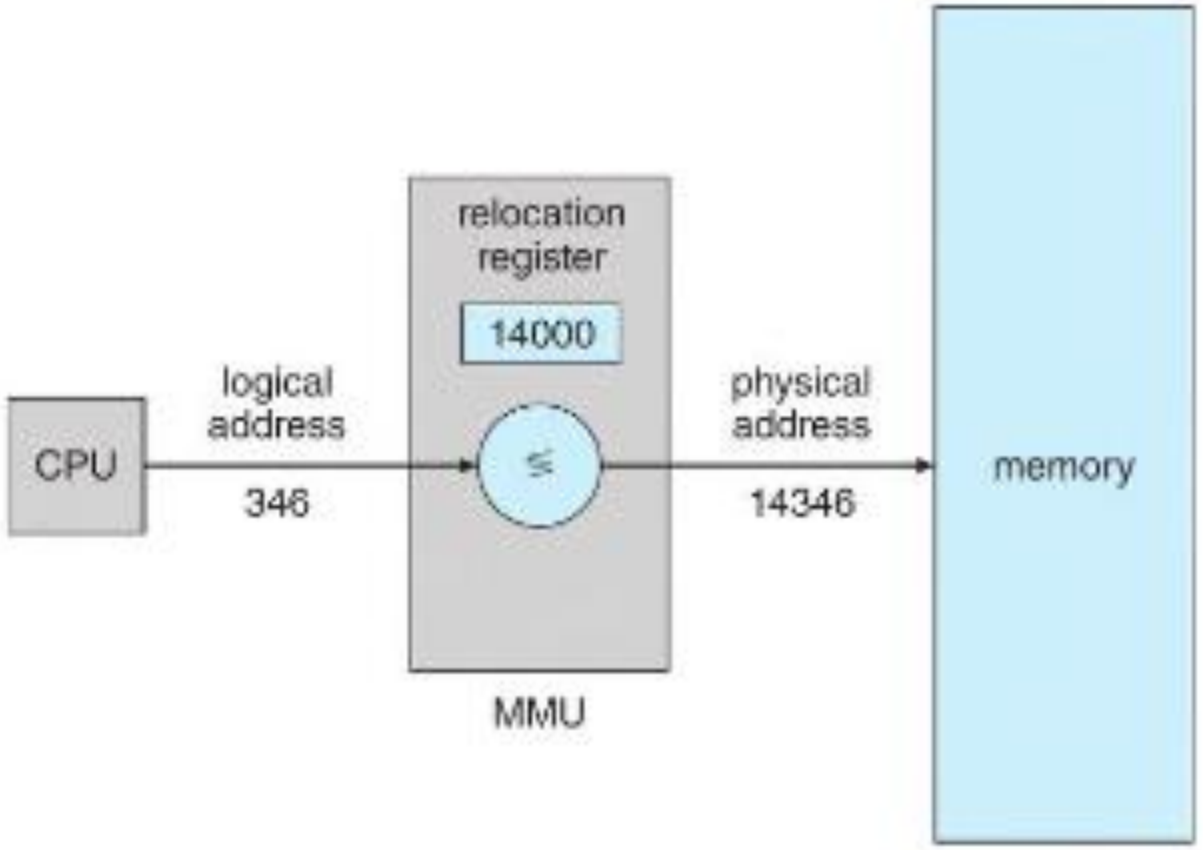
- **For example, the base register is now called a relocation register.**
  - The relocation register is a **special part of the MMU** that holds a fixed offset (a number). This offset helps the MMU shift (or relocate) the virtual address to the correct physical address.
  - This offset is the difference between where the program thinks it's running and where it's actually located in physical memory.

Memory Management Unit (MMU)

- **Relocation Register - Example**
  - Imagine a process starts its virtual address at 0, but in real memory, it starts at physical address 5000.
  - The relocation register will hold the value 5000.
  - So, if the program asks for data at virtual address 20, the MMU adds 5000 to this, making the actual physical address 5020.

Memory Management Unit (MMU)

- Relocation Register - Example



Static Loading

- When a program is about to run, static loading means that **everything the program needs (code, data, etc.) is loaded into memory** before it **starts**.
- The memory locations for all components are set in advance, and they **don't change while the program is running**.

Pros: Fast access to everything since it's already in memory.

Cons: This can waste memory if not all components are actually used during the program's run.

Example: Imagine opening a big book. You copy the entire book onto your desk even if you might only read a few pages.

Static Linking

- Static linking happens when the necessary libraries are added directly into the program's file when it is created.
- The final program file contains everything it needs to run on its own, so it **doesn't need to find any external libraries at runtime.**
- **Pros:** It makes the program independent and portable
- **Cons:** The file becomes larger, and if many programs use the same library, each will have its own copy, leading to redundancy.

Dynamic Linking

- With dynamic linking, the program does not include the libraries in its final file. Instead, it **connects to these libraries while running**.
- This allows **many programs to share a single copy of a library**, saving memory and reducing file size.
- Pros: Saves space and avoids redundancy.
- Cons: The program depends on these libraries being available on the system during runtime

Dynamic Loading

- In dynamic loading, the **program loads components into memory only when they are needed during execution.** It doesn't load everything upfront.
- With dynamic loading, a routine is not loaded until it is called. All routines are kept on disk in a relocatable load format.
- Then the relocatable linking loader is called to load the desired routine into memory and to update the program's address tables to reflect this change.

Dynamic Loading

- The advantage of dynamic loading is that a routine is loaded only when it is needed.
- Dynamic loading does not require special support from the operating system.
- Better memory management, as unused parts don’t take up space.
- It can slow down the program slightly if it has to load new components while running.